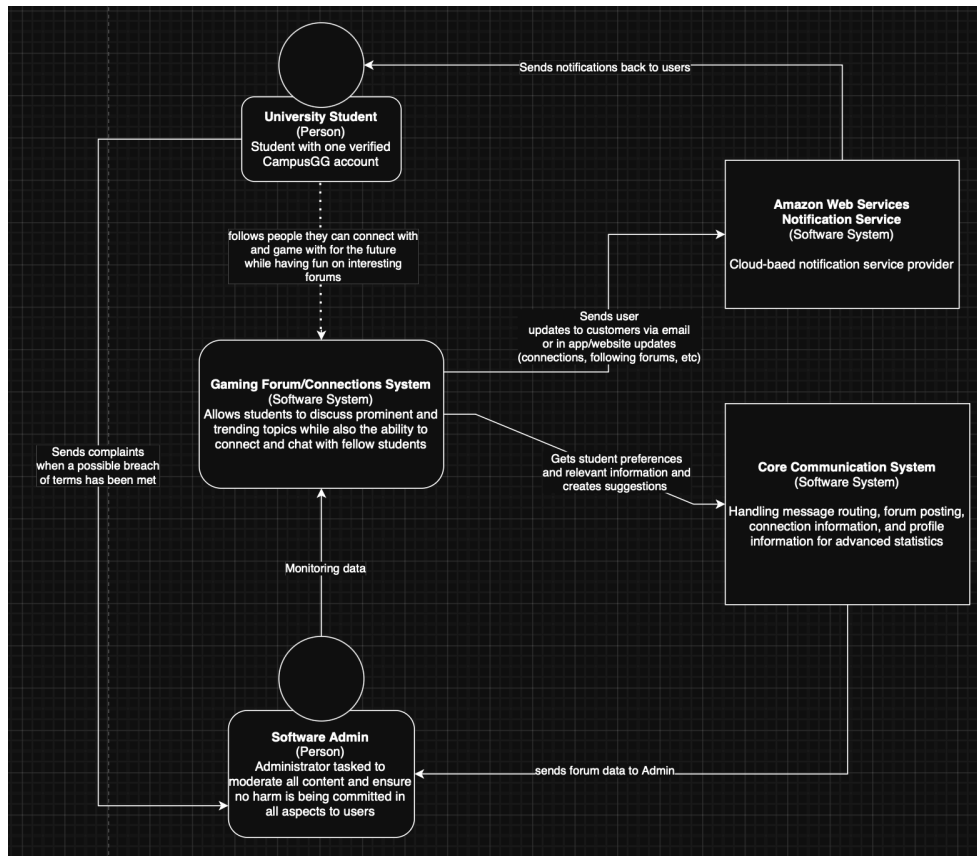


ECE 49595 SD I – Design Document

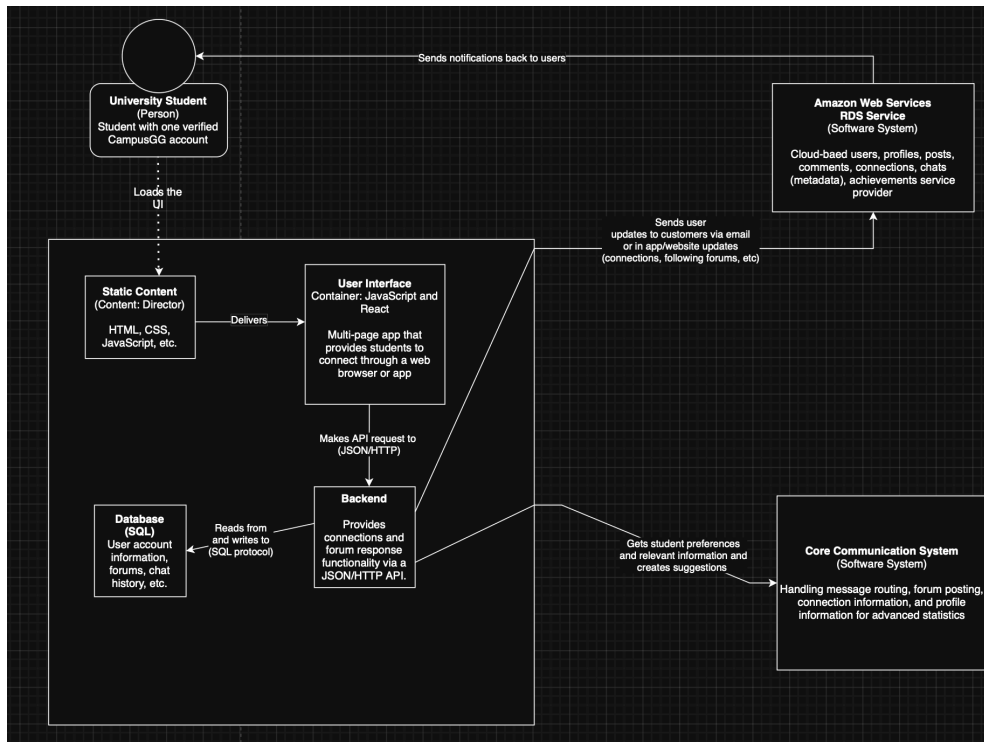
Spring 2026

Design Diagrams

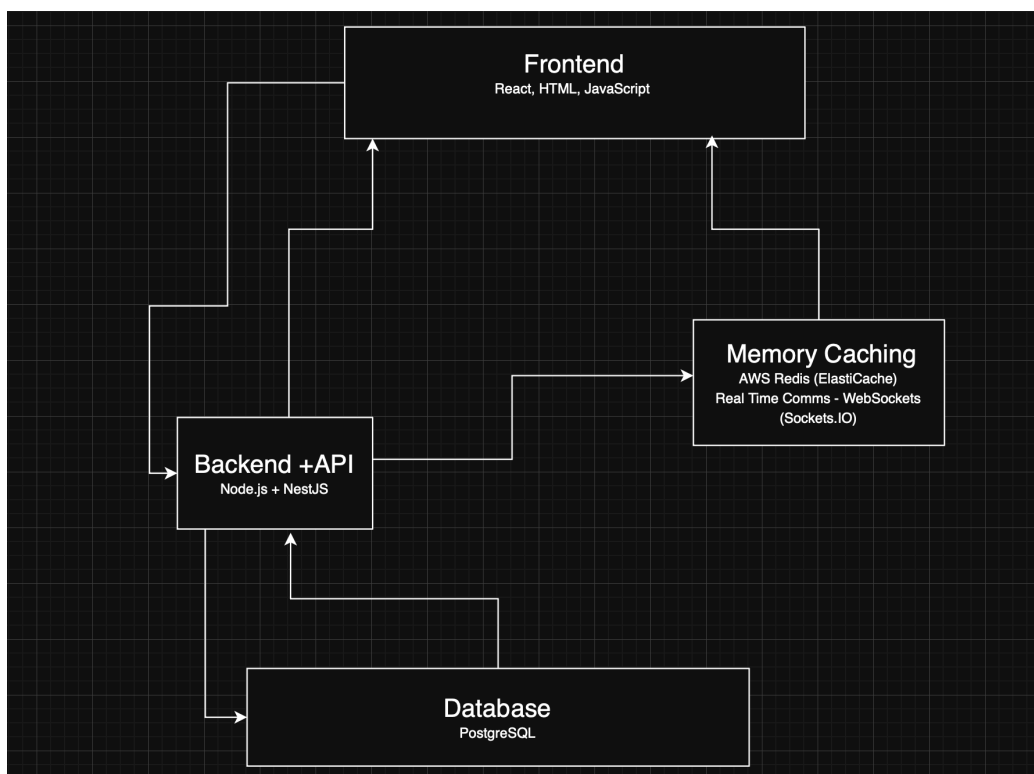
- System Context Diagram



Container Diagram



Technology Stack



These technologies form the primary architecture of the CampusGG platform.

Frontend (UI & Client Interface):

React: Component-based UI framework used to build the main web interface (Lobbies, Forums, Profiles).

HTML & JavaScript: Core technologies utilized for page structure, layout, and client-side logic.

Backend & APIs:

Node.js & NestJS: Backend framework used to build structured APIs, authentication services, and WebSocket endpoints.

Database:

PostgreSQL: Primary relational database for storing users, forum posts, connections, and achievements.

Memory Caching & Real-Time Communication:

AWS Redis: Utilized for session memory caching, presence tracking ("currently playing"), and feed caching.

WebSockets: Enables real-time communication features like instant chat, live lobby updates, and user presence without requiring page refreshes.

Figma

- <https://www.figma.com/design/UcU5BIya0YiAtiSwhQ2bTj/Candidate-Designs?node-id=0-1&t=D6RpvvWp45MfixpG-1>

Design Trade

Third Party API Integration - Justin Samuel

Design #001 - Direct Live API Calls

CampusGG directly calls the third-party game stats API whenever user data is needed. The backend sends a request to the external service in real time, receives the response, and immediately passes the processed data back into the platform. This is the simplest and most straightforward design

Design #002 - Cached API Integration Layer

CampusGG calls the third-party API through a backend integration service that stores recently retrieved results in a cache for a limited period of time. If the same user's data is requested again within that period, the system serves the cached result instead of making another live API call.

Design #003 — Scheduled Sync + Local Database Mirror

CampusGG periodically pulls third-party game data in the background and stores normalized results in its own database. The app mostly reads from the local copy instead of calling the external API during normal user activity.

Alternative	Reliability (35%)	Response Time (20%)	Scalability (15%)	Security (20%)	Rate-Limit Handling (10%)	Total
Design 001: Real-Time External API Fetching	5	5	4	7	3	5.1
Design 002:	8	7	9	8	8	7.95

Cached API Gateway Layer						
Design 003: Periodic Data Synchroniz ation System	9	8	7	6	9	7.9

The cached API integration layer is the best overall choice because it balances performance, reliability, and implementation complexity better than the other alternatives. Direct live API calls are easy to build, but they depend too heavily on outside API availability and are more likely to run into latency and rate-limit issues. The scheduled sync and local mirror approach performs very well, but it is more complex than what CampusGG currently needs. The cached API layer gives the team a practical middle-ground solution by reducing repeated external requests, improving response times, and lowering the risk of hitting API limits while still keeping the design manageable for development and maintenance.

Forum Content Management Service - Jingwen Huang

Design #001 - Simple Forum Service

This design uses one backend service to handle all forum functions, including creating, viewing, replying to, editing, and deleting posts. All forum data is stored in PostgreSQL, making this option easy to build for an early version but harder to scale as features grow.

Design #002 - Modular Forum Service

This design separates the forum into modules for posts/replies, moderation, and search/filtering while still running within one backend system. It is more organized and maintainable than the simple design, making it easier to add future features, though it requires more planning and testing.

Design #003 - Event-Based Forum Service

This design uses events when forum actions occur, such as creating a post, so other services can handle moderation, notifications, or analytics. It provides strong scalability and future flexibility, but it is the most complex to build, manage, and debug.

Design	Performance	Scalability	Privacy and Access Control	Accessibility and Usability	Development Complexity	Weighted Total Score
Simple Forum Service	7	5	7	8	9	7.2
Modular Forum Service Design	8	8	8	8	7	7.85
Event-Based Forum Service	9	10	8	7	4	7.65

The chosen solution is the Modular Forum Service Design because it received the highest weighted total score. It provides the best overall balance between performance, scalability, privacy/access control, usability, and development effort. The Simple Forum Service would be easier to build at first, but it would become harder to maintain as CampusGG grows. The Event-Based Forum Service offers the strongest scalability, but its added complexity would increase implementation risk for the current project scope. Therefore, the modular design is the best trade-off because it supports future growth while still remaining realistic for the team to build and test.

University Verification & Authentication - Tianyu Du

Design #001 - Custom .edu Email OTP Verification

A custom backend service that generates and sends a one-time password directly to a user's .edu email address via SMTP. This approach provides full control over our data schema and avoids third-party dependencies, though it requires meticulous internal security management.

Design #002 - Third-Party Identity Provider (e.g., Firebase Auth / Auth0)

Outsourcing credential management and university email verification to a managed authentication service. This ensures hardened, production-ready security and automatic scaling, but introduces ongoing operational costs and potential vendor lock-in.

Design #001 - Direct University SSO Integration (SAML/Shibboleth)

Partnering directly with university IT departments to allow students to log in using their existing campus credentials. While offering the most frictionless user experience, it presents insurmountable scaling challenges due to the overhead of negotiating individual institutional agreements.

Evaluation Table

Criterion	Weight	A1: Custom OTP	✓ A2: Third-Party IdP (Best Option)	A3: Direct SSO
Integration Complexity	20%	8 (1.60)	9 (1.80)	2 (0.40)
Security & Compliance	30%	6 (1.80)	9 (2.70)	10 (3.00)
Operational Cost	15%	10 (1.50)	4 (0.60)	7 (1.05)
User Onboarding Friction	20%	7 (1.40)	8 (1.60)	9 (1.80)

Scalability	15%	6 (0.90)	10 (1.50)	3 (0.45)
Total Weighted Score	100%	7.20	8.20	6.70

(Scores are on a 1–10 scale. The numbers in parentheses represent the weighted scores.)

The Third-Party Identity Provider was selected because the security of student data and the speed of integration are paramount for this subsystem. While building a custom OTP flow would save operational costs, the engineering overhead and severe security risks associated with rolling our own token generation are too high for our current timeline. Outsourcing credential management to a managed service like Firebase Auth or Auth0 ensures hardened, production-ready security and seamless scalability. This trade-off accepts higher ongoing costs in exchange for guaranteed security and allows the team to focus development hours on core platform features rather than reinventing authentication.